When Reflection Helps: Utility-Guided Advantage Rescaling for Efficient Reasoning

Jungyu Lee, Kunhui Lee, Dongwook Min, Jeonhwi Kang, Seung-Hoon Na*
UNIST
{steamy13133, nash}@unist.ac.kr

Abstract

Large reasoning models have recently demonstrated emergent "aha moments" that enable reflective reasoning. However, a non-trivial portion of reflective steps are "low-utility," where reasoning errors are not corrected, or even correct reasoning trajectories are shifted toward incorrect ones. To address low-utility reflective continuation, this paper proposes **RUAR**, a utility-guided advantage rescaling method for reflective reasoning that estimates step-level utility from changes in correct-answer probability, without human process labels or a separately learned process reward model. RUAR augments outcome-supervised RL by rescaling the token-level advantages within each reflective reasoning step according to the estimated utility of the reflective continuation, rather than applying a uniform trajectory-level advantage across the reasoning trace. The utility is estimated through **answer-forcing utility estimation**, which measures how the model's correct-answer probability changes before and after the reflective step by forcing answer generation from both reasoning states using answer-forcing prompts. Furthermore, RUAR introduces an **overthinking-mitigating rescaling mechanism**, where utility-guided rescaling is no longer applied from the earliest answer-ready reflection step onward. Experiments demonstrate that RUAR improves the accuracy–efficiency trade-off, substantially reducing generated token usage while often preserving or improving final-answer accuracy. Our code is available at https://github.com/won12055/RUAR.

1 Introduction

Recently, large reasoning models (LRMs) have demonstrated strong performance on complex reasoning tasks by leveraging increased inference-time computation through extended intermediate reasoning (Snell et al., 2025; Guo et al., 2025). A key factor underlying these capabilities is the emergence of "aha moments," a form of reflective reasoning in which models revisit, verify, and reorganize an existing reasoning trajectory, thereby recovering from earlier errors and eventually reaching correct solutions to challenging problems through iterative reflection (Guo et al., 2025).

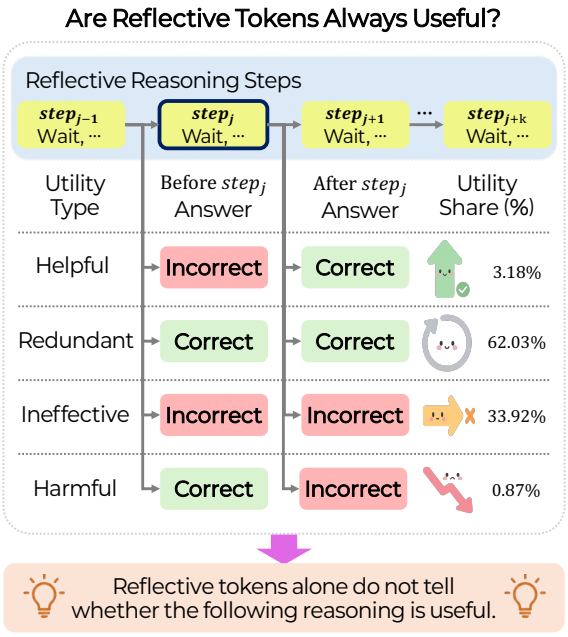

Figure 1: Utility distribution over 8,590 extracted *Wait* reflective reasoning steps from Qwen3-8B MATH500 base-model traces generated under greedy decoding with a 32,768-token response limit. For each step, answer-forcing generates final answers from the before-step and after-step partial traces, and each answer is compared with the gold answer to identify whether the step is *helpful*, *redundant*, *ineffective*, or *harmful*.

Such reflective behavior is often associated with reflective tokens such as *Wait* and *Alternatively*, which allow models to revisit and extend an existing reasoning trajectory (Guo et al., 2025). However, contrary to the expected self-correction effect of reflective continuation, reflection is not always

*Corresponding author

useful (Huang et al., 2024; Wang et al., 2025). As illustrated in Figure 1, only 3.18% of extracted *Wait* steps are helpful, while 96.82% are redundant, ineffective, or harmful. We refer to these latter three categories as **low-utility** reflection: continuation that consumes additional reasoning tokens without improving the probability of producing the correct answer. Thus, reflective tokens provide useful boundaries for reflective reasoning steps, but they are not reliable utility signals.

Prior work has mainly handled low-utility reflective continuation by reducing or stopping reflection at a coarser level. Some methods suppress reflective tokens during decoding to disable explicit self-reflection (Wang et al., 2025). Others observe that later reflection, especially after a candidate answer has appeared, is often confirmatory, and therefore truncate subsequent reflection or stop reasoning early (Kang et al., 2026). More general efficient-reasoning methods prune redundant reasoning chunks using attention (Choi et al., 2025) or verifier signals (Lin et al., 2026), or train models with length-oriented objectives (Luo et al., 2025) and token-budget constraints (Hou et al., 2026). These approaches can reduce token usage, but they do not directly determine whether a particular reflective reasoning step is helpful, redundant, ineffective, or harmful.

Motivated by the need to suppress low-utility reflection while preserving corrective high-utility reflection, we propose RUAR, *reflective reasoning with utility-guided advantage rescaling*. Instead of using a fixed trace-level advantage score uniformly across all tokens in a reasoning trace during reinforcement learning (RL) training, RUAR rescales token-level advantage signals for each reflective reasoning step according to its estimated utility. In particular, RUAR performs **answer-forcing utility estimation**: for each reflective reasoning step, RUAR applies *answer-forcing prompts* to both the "before-step" and "after-step" partial traces, compares the resulting forced-answer correctness rates, and estimates how the correct-answer probability changes before and after the reflective continuation.

Furthermore, once the model reaches the earliest answer-ready reflection step, additional reflective continuation can become redundant and contribute to overthinking behavior. To mitigate this issue, RUAR further introduces an **overthinking-mitigating rescaling method**, where utility-guided rescaling is no longer applied from the earliest answer-ready reflection step onward, and a fixed sign-dependent rescaling strategy is instead employed depending on whether the original advantage is positive or negative. As a result, RUAR enables step-level credit assignment without requiring human process labels, learned process reward models, or step-level annotations.

Our contributions are threefold. First, we empirically show that reflective reasoning steps differ substantially in their self-correction effectiveness, and that a non-trivial portion of reflective continuation suffers from low-utility behavior. Second, we propose RUAR, a utility-guided advantage rescaling method for reflective reasoning, consisting of answer-forcing-driven utility estimation and an overthinking-mitigating rescaling mechanism. Third, experiments on open-weight reasoning models demonstrate that RUAR improves the accuracy–efficiency trade-off, substantially reducing generated token usage while often preserving or improving final-answer accuracy.

2 Preliminaries

2.1 Reasoning by Multiple Reflection Steps

Given a question (or prompt) x, the policy of an LRM, denoted by π_θ, generates a full response $\boldsymbol{y} = (y_1, \ldots, y_T)$, where T is the total number of generated tokens. The full response is decomposed into a reasoning segment and an answer segment, written as $\boldsymbol{y} = [\boldsymbol{y}_r; \boldsymbol{y}_a]$, where $\boldsymbol{y}_r = (y_1, \ldots, y_{T_r})$ denotes the reasoning segment and $\boldsymbol{y}_a = (y_{T_r+1}, \ldots, y_T)$ denotes the answer segment. The two segments are separated by the closing thinking tag $\langle$/think$\rangle$.

During reflective reasoning, the reasoning segment $\boldsymbol{y}_r$ exhibits an extended thinking process through the frequent use of reflective tokens such as *Wait* and *Alternatively*. By treating configurable subsets of reflective tokens $\mathcal{R}_{\text{start}}$ and $\mathcal{R}_{\text{end}}$ as target step-start and step-end delimiters, respectively, while including the corresponding start token within each reflective step, we extract a sequence of reflective reasoning steps $(\boldsymbol{s}_1, \ldots, \boldsymbol{s}_m)$ from $\boldsymbol{y}_r$, where $\boldsymbol{s}_j = y_{a_j}, \ldots, y_{e_j}$ denotes the j-th reflective reasoning step, spanning from the a_j-th token to the e_j-th token in $\boldsymbol{y}_r$. These extracted reflective reasoning steps need not cover the entire reasoning segment. The starting token of each reflective step, $y_{a_j} \in \mathcal{R}_{\text{start}}$, is always the reflective token used as the target step-start delimiter. Each step ends immediately before the next delimiter in $\mathcal{R}_{\text{end}}$. If no later endpoint delimiter exists, the final reflective step ends before the closing thinking tag

⟨/think⟩. Appendix G provides a method-aligned inventory of these reflective tokens and discusses the rationale for the delimiter choices.

As discussed in Section 1, there is no guarantee that reflective reasoning steps consistently improve answer correctness throughout the reasoning trajectory. This motivates estimating the utility of each reflective step based on its before/after effect on correct-answer probability, rather than assigning static rewards or penalties solely based on the presence of reflective tokens.

2.2 Length-Regularized RLOO Trainer

For each prompt x, we sample N rollouts $\{\boldsymbol{y}^{(i)}\}_{i=1}^{N}$ from the current policy and assign each rollout a terminal reward R_i. Our base trainer uses a REINFORCE leave-one-out (RLOO) advantage estimator (Ahmadian et al., 2024). For rollout i, the leave-one-out baseline and rollout-level advantage are

$$\bar{R}_{-i} = \frac{1}{N-1}\sum_{n\neq i} R_n, \qquad \hat{A}_i = R_i - \bar{R}_{-i}.$$

This rollout-level signal is broadcast to response tokens, yielding the base token-level advantage signal $A_{i,t}$ that RUAR later rescales before the final policy update.

Following prior efficient-reasoning work (Arora and Zanette, 2026), we use a length-regularized terminal reward in the RLOO trainer. The exact reward definition is provided in Appendix E.

3 Method

3.1 Overview

As discussed in Section 2.2, standard outcome-supervised RL assigns the same trace-level supervision signal to a full response by using identical advantage scores across all tokens in a reasoning trace, without considering that reflective reasoning steps may differ substantially in whether they correct, repeat, or derail the solution trajectory. Instead of using trace-constant advantages $A_{i,t}$, RUAR estimates the utility of each reflective reasoning step $\boldsymbol{s}_j$ from its before/after effect on correct-answer probability under answer-forcing prompts, and uses this utility to reshape token-level advantage signals during RL training.

In addition, to mitigate overthinking, RUAR further identifies the *earliest answer-ready step*: before this step, it applies utility-guided rescaling to reflective reasoning steps, while from this step onward, it applies fixed overthinking-mitigating rescaling to redundant post-ready continuation. For readability, we use simplified notation for step-level quantities within a fixed rollout, while Algorithm 1 uses rollout-indexed notation for the full training procedure.

3.2 Answer-Forcing Utility Estimation

With multiple reflective reasoning steps defined in Section 2.1, RUAR performs answer-forcing utility estimation to determine whether each reflective step improves correct-answer probability. For the j-th reflective reasoning step $\boldsymbol{s}_j$ in $\boldsymbol{y}$, defined as $\boldsymbol{s}_j = y_{a_j}, \ldots, y_{e_j}$, we define the corresponding *before-step* and *after-step* partial traces as

$$\boldsymbol{s}_j^- = \boldsymbol{y}_{1:a_j-1}, \qquad \boldsymbol{s}_j^+ = \boldsymbol{y}_{1:e_j}. \tag{1}$$

Here, $\boldsymbol{y}_{m:n}$ denotes the span of response tokens starting from y_m and ending at y_n.

Given a before-step or after-step partial trace $\boldsymbol{y}_{1:q}$ (i.e., either $\boldsymbol{s}_j^-$ or $\boldsymbol{s}_j^+$), we apply an *answer-forcing prompt* $\mathcal{A}$ to evaluate whether $\boldsymbol{y}_{1:q}$ contains sufficient reasoning information for the model π_θ to generate the correct final answer when explicitly forced to answer, as follows:

```
\n**Final Answer**\n\boxed
```

We then sample K short *final-answer continuations* from the current policy. Let $o_{j,k}^-$ and $o_{j,k}^+$ denote the k-th continuations sampled from $\boldsymbol{s}_j^- \oplus \mathcal{A}$ and $\boldsymbol{s}_j^+ \oplus \mathcal{A}$, respectively, where $\oplus$ denotes string concatenation. Let $\mathsf{correct}$ be a *rule-based final-answer verifier* satisfying $\mathsf{correct}(\cdot) \in \{0, 1\}$. The estimated correct-answer probabilities for the before-step and after-step partial traces are computed as follows:

$$\begin{aligned} p_j^- &= \frac{1}{K}\sum_{k=1}^{K} \mathsf{correct}(x, \boldsymbol{s}_j^- \oplus \mathcal{A} \oplus o_{j,k}^-), \\ p_j^+ &= \frac{1}{K}\sum_{k=1}^{K} \mathsf{correct}(x, \boldsymbol{s}_j^+ \oplus \mathcal{A} \oplus o_{j,k}^+). \end{aligned} \tag{2}$$

where p_j^- and p_j^+ denote the *before-step* and *after-step correct-answer probabilities*, respectively.

The utility of the reflective reasoning step $\boldsymbol{s}_j$ is then defined as

$$u_j = p_j^+ - p_j^-. \tag{3}$$

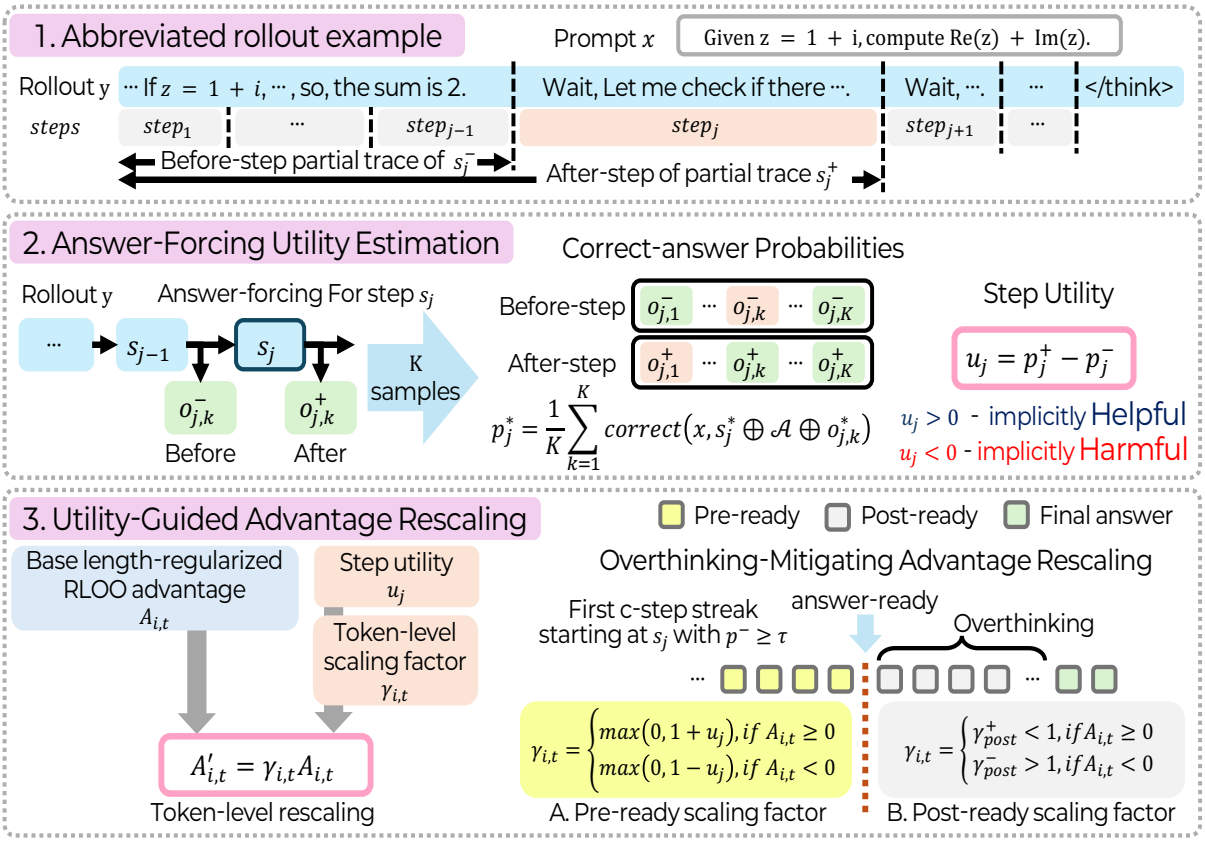

Figure 2: An overall pipeline of RUAR on an abbreviated rollout. 1 Given a full rollout $\boldsymbol{y}$, RUAR first segments $\boldsymbol{y}$ into m reflective reasoning steps, $(\boldsymbol{s}_j)_{j=1}^{m}$. The *before-step* and *after-step* partial traces of $\boldsymbol{s}_j$ correspond to the partial traces immediately before and after $\boldsymbol{s}_j$, respectively, as defined in Eq. 1. 2 Answer-forcing utility estimation appends the answer-forcing prompt $\mathcal{A}$ to both the before-step and after-step partial traces, i.e., $\boldsymbol{s}_j^-$ and $\boldsymbol{s}_j^+$, and immediately generates a set of answer completions, denoted by $\left\{o_{j,k}^-\right\}_{k=1}^{K}$ and $\left\{o_{j,k}^+\right\}_{k=1}^{K}$, respectively. The corresponding correct-answer probabilities, denoted by p_j^- and p_j^+ and computed using Eq. 2, are then used to estimate the utility of $\boldsymbol{s}_j$, where $u_j = p_j^+ - p_j^-$. 3 To mitigate overthinking behavior, utility-guided advantage rescaling is applied only to pre-ready reflective reasoning steps, where the model is not yet sufficiently ready to generate the correct answer. For the remaining post-ready reflective reasoning steps, a simpler constant-based scaling strategy is applied, reducing the likelihood of generating additional reflective continuation, as defined in Eq. 7.

The verifier correct evaluates only the sampled final-answer continuations and does not provide human process labels or step-level supervision. See Appendix F for implementation details of the answer-forcing prompt and verification procedure.

3.3 Utility-Guided Advantage Rescaling

Let $A_{i,t}$ denote the base token-level advantage signal assigned by the length-regularized RLOO trainer before the final policy update. RUAR constructs a token-level scaling factor $\gamma_{i,t}$ and computes the rescaled advantage signal as follows:

$$A'_{i,t} = \gamma_{i,t} A_{i,t}. \tag{4}$$

The RL trainer subsequently applies its standard advantage normalization procedure to $A'_{i,t}$ before the PPO-style policy update.

For a token t belonging to a reflective reasoning step $\boldsymbol{s}_j$ to which utility-guided rescaling is applied, RUAR computes the scaling factor based on the estimated utility u_j of the reflective step:

$$\gamma_{i,t} = \begin{cases} \max(0, 1 + u_j), & A_{i,t} \geq 0, \\ \max(0, 1 - u_j), & A_{i,t} < 0. \end{cases} \tag{5}$$

Thus, when $A_{i,t}$ is positive, helpful reflective steps receive stronger positive credit, while harmful reflective steps receive weaker positive credit. Conversely, when $A_{i,t}$ is negative, helpful reflective

steps are penalized less, whereas harmful reflective steps are penalized more strongly.

3.4 Overthinking-Mitigating Rescaling

The utility-guided rescaling rule in Section 3.3 assigns step-specific credit according to estimated step utility. However, once a reasoning trajectory is already likely to produce the correct final answer, additional reflective continuation can become redundant. RUAR therefore applies utility-guided rescaling only before the model becomes answer-ready, and switches to fixed post-ready rescaling afterward.

3.4.1 Pre-Ready and Post-Ready Reflection

We divide reflective reasoning into two stages: pre-ready and post-ready.

Pre-ready stage. At the pre-ready stage, the current reasoning trajectory is not yet sufficiently reliable, and further exploration or self-correction may still be required before the model becomes ready to generate the correct answer.

Post-ready stage. At the post-ready stage, the reasoning trajectory has become sufficiently reliable, and the model is already likely to generate the correct answer. Additional reflective continuation after this point is therefore more likely to become redundant and contribute to overthinking behavior.

To identify the transition between these stages, we define a reflective step as *near answer-ready* if its before-step correct-answer probability is at least a threshold τ. If the j-th reflective step and its following $c-1$ consecutive reflective steps are all near answer-ready, then the j-th step is identified as the earliest answer-ready reflection step, referred to simply as the *answer-ready step*. Formally, the index b of the answer-ready step is

$$b = \min\{j : p_r^- \geq \tau \quad \text{for } r = j, \ldots, j+c-1\}. \tag{6}$$

If no such consecutive run exists, we set $b = m+1$.

Using the answer-ready step $\boldsymbol{s}_b$, we define pre-ready and post-ready reflective reasoning steps as follows: **Pre-ready reflective steps**: $\boldsymbol{y}^{\text{pre}} = (\boldsymbol{s}_1, \ldots, \boldsymbol{s}_{b-1})$. **Post-ready reflective steps**: $\boldsymbol{y}^{\text{post}} = (\boldsymbol{s}_b, \ldots, \boldsymbol{s}_m)$.

Here, $\boldsymbol{y}^{\text{pre}}$ contains reflective steps before the answer-ready step, while $\boldsymbol{y}^{\text{post}}$ contains the answer-ready step itself and later reflective continuation, i.e., from the answer-ready step onward, inclusive.

During training, once the answer-ready step is detected, RUAR stops computing additional answer-forcing utility estimates for the remaining reflective continuation within the rollout. A concrete probing example is provided in Appendix M, while Appendix H further analyzes the behavior of answer-ready step detection.

3.4.2 Pre-Ready and Post-Ready Rescaling

Using the pre-ready and post-ready reflective reasoning steps defined above, RUAR specifies which scaling factor $\gamma_{i,t}$ is used in Eq. 4 for each token region. For pre-ready tokens $y_t \in \boldsymbol{y}^{\text{pre}}$, the scaling factor $\gamma_{i,t}$ follows the utility-guided rule in Section 3.3. For post-ready continuation, RUAR no longer uses the estimated step utility. Once the answer-ready step $\boldsymbol{s}_b$ is detected, fixed post-ready rescaling is applied to all remaining reasoning-segment tokens from the start of $\boldsymbol{s}_b$ onward:

$$\gamma_{i,t} = \begin{cases} \gamma_{\text{post}}^+, & A_{i,t} \geq 0, \\ \gamma_{\text{post}}^-, & A_{i,t} < 0, \end{cases} \tag{7}$$

where $\gamma_{\text{post}}^+ < 1$ and $\gamma_{\text{post}}^- > 1$.

Tokens outside both pre-ready reflective reasoning steps and this post-ready continuation use the default scaling factor $\gamma_{i,t} = 1$.

3.5 Policy Optimization

After standard advantage normalization, we use the normalized RUAR-adjusted advantage signal in the same PPO-style policy objective as the base trainer:

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}_{i,t}\big[\min(r_{i,t}A'_{i,t}, \text{clip}_\epsilon(r_{i,t})A'_{i,t})\big]. \tag{8}$$

where

$$\text{clip}_\epsilon(r) = \text{clip}(r, 1-\epsilon, 1+\epsilon). \tag{9}$$

Here, $r_{i,t}$ denotes the policy likelihood ratio. RUAR preserves the original outcome-based RL trainer, including rollout generation, terminal outcome scoring, and terminal reward computation. The additional component introduced by RUAR is a step-level rescaling layer that applies utility-guided scaling before the answer-ready step and fixed post-ready scaling afterward.

Method	GSM8K		MATH500		AIME2024		HMMT2025		GPQA		ARC-C		CSQA		Weighted Avg.	
	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.
DeepSeek-R1-Distill-Qwen-1.5B																
Base	76.4	842	63.8	3,542	16.7	7,626	6.7	6,422	27.4	3,146	56.2	787	46.2	640	58.4	1,259
O1-Pruner	75.6	879	64.4	3,647	10.0	6,802	3.3	6,496	27.9	3,255	56.1	772	45.4	673	58.0	1,287
ThinkPrune	**83.5**	821	76.0	2,571	20.0	5,841	3.3	6,064	22.8	2,769	58.3	660	45.5	558	62.0	1,057
DEER	68.2	754	64.4	2,964	23.3	11,387	10.0	11,332	33.0	9,507	**59.6**	1,145	47.0	953	57.5	1,686
ARLCP	76.1	698	69.8	3,016	16.7	7,684	**13.3**	6,469	25.9	3,070	55.8	724	46.1	623	58.8	1,134
PRIME	63.3	1,942	54.0	4,059	13.3	6,954	3.3	7,575	24.4	3,896	56.2	1,781	45.8	1,293	53.2	2,117
TLMRE	83.4	509	78.8	1,686	**30.0**	**4,495**	10.0	**4,435**	26.4	2,383	58.7	579	**48.2**	432	**63.4**	774
RUAR	82.2	**464**	**79.4**	**1,545**	26.7	5,310	**13.3**	5,599	**34.5**	**2,002**	58.5	**492**	46.5	**374**	63.0	**702**
DeepSeek-R1-Distill-Qwen-7B																
Base	89.8	998	85.0	3,569	33.3	9,311	23.3	9,959	44.7	5,818	82.1	677	65.4	657	77.7	1,437
O1-Pruner	89.3	1,128	86.8	3,369	36.7	8,934	20.0	11,341	40.6	5,418	81.7	716	**66.9**	658	77.9	1,452
ThinkPrune	**92.1**	910	**90.0**	2,553	53.3	7,234	13.3	8,945	43.1	4,174	83.5	568	65.9	544	79.5	1,144
DEER	90.5	723	87.4	2,470	46.7	9,781	23.3	11,546	44.2	6,697	81.3	551	65.0	545	78.0	1,222
ARLCP	90.6	1,353	85.0	3,349	40.0	9,153	16.7	10,478	**46.2**	5,714	83.1	777	65.9	689	78.5	1,550
PRIME	89.4	2,051	83.8	4,352	40.0	10,663	**26.7**	11,610	45.7	7,533	81.9	1,142	65.8	1,023	77.7	2,153
TLMRE	86.7	765	89.6	2,199	40.0	8,672	16.7	**8,437**	**46.2**	4,418	**84.5**	555	65.6	488	78.1	1,060
RUAR	90.5	**674**	**90.0**	**2,100**	**60.0**	**7,034**	16.7	8,711	44.7	**3,407**	84.4	**471**	66.6	**413**	**79.6**	**926**
Qwen3-1.7B																
Base	89.6	1,936	86.4	4,896	36.7	12,388	23.3	13,399	34.5	7,718	88.3	1,122	**74.8**	1,092	81.6	2,225
O1-Pruner	**91.0**	1,957	87.4	4,797	36.7	12,784	20.0	13,174	38.1	7,449	88.8	1,123	73.9	1,087	**82.2**	2,208
ThinkPrune	90.2	1,733	86.0	4,435	40.0	12,670	16.7	13,554	38.6	8,304	**89.1**	1,114	74.2	1,022	82.0	2,121
DEER	89.3	**1,080**	86.4	**3,248**	40.0	**10,543**	13.3	12,713	36.0	**4,388**	87.7	**748**	72.2	**694**	80.7	**1,417**
ARLCP	89.7	1,724	86.6	4,639	33.3	13,279	**26.7**	13,057	**39.1**	7,204	**89.1**	1,036	72.7	1,010	81.5	2,070
PRIME	90.3	1,687	86.4	5,021	**46.7**	11,777	16.7	13,794	36.0	7,528	88.9	1,046	74.5	1,007	82.0	2,113
TLMRE	90.0	1,094	87.2	3,554	26.7	12,191	23.3	**12,448**	35.0	6,689	88.4	838	74.1	806	81.6	1,621
RUAR	90.0	1,137	**87.6**	3,450	36.7	12,209	13.3	13,096	34.0	6,559	87.5	814	74.2	780	81.4	1,607
Qwen3-8B																
Base	95.3	2,184	92.0	4,852	63.3	11,059	30.0	14,042	55.3	8,525	93.8	1,340	83.7	1,332	88.9	2,447
O1-Pruner	95.6	2,174	92.2	5,024	60.0	11,622	**43.3**	13,744	57.9	8,673	95.7	1,350	84.2	1,325	89.9	2,472
ThinkPrune	95.3	1,599	93.2	3,970	**70.0**	10,018	40.0	12,262	56.9	7,434	**96.3**	1,136	83.5	1,089	89.9	1,989
DEER	95.5	1,079	92.0	3,208	**70.0**	10,081	36.7	**11,676**	49.2	**4,271**	94.5	**710**	82.5	**723**	88.7	**1,395**
ARLCP	95.6	1,835	91.6	4,332	60.0	10,083	30.0	13,775	58.9	7,571	96.0	1,193	83.5	1,166	89.6	2,152
PRIME	94.7	3,363	89.2	6,797	60.0	12,813	30.0	14,953	53.8	11,067	95.7	1,841	84.2	1,808	89.0	3,404
TLMRE	95.6	1,096	93.0	3,029	63.3	10,196	30.0	12,045	53.3	5,906	95.7	850	84.5	892	89.8	1,539
RUAR	**95.9**	**950**	**93.6**	**2,920**	60.0	**9,123**	30.0	13,042	**59.9**	5,869	95.4	754	**85.1**	761	**90.3**	1,420

Table 1: Main results under the shared greedy 16K evaluation protocol. Acc. is reported in percent and Tok. is the mean response length. First-ranked cells are bold with green shading, and second-ranked cells use yellow shading (higher is better for Acc.; lower is better for Tok.).

4 Experiments

4.1 Experimental Setup

Backbones and baselines. We evaluate RUAR as a post-training method for improving the accuracy–efficiency trade-off of reasoning models. Our main comparison spans four open-weight reasoning backbones: DeepSeek-R1-Distill-Qwen-1.5B and DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025), and Qwen3-1.7B and Qwen3-8B (Yang et al., 2025).

For each backbone, we compare eight methods: the original Base model, TLMRE (Arora and Zanette, 2026), PRIME (Cui et al., 2026), O1-Pruner (Luo et al., 2025), ThinkPrune (Hou et al., 2026), DEER (Yang et al., 2026), ARLCP (Yu et al.,

2026b), and RUAR. These baselines cover complementary efficiency paradigms: length-harmonizing fine-tuning (O1-Pruner), reinforcement-learning-based long-chain-of-thought pruning (ThinkPrune), training-free dynamic early exit (DEER), adaptive reflection with a length-coordinated penalty (ARLCP), implicit process reward modeling (PRIME), and trajectory-level length-regularized RL (TLMRE).

Training and checkpoint selection. The RUAR policies are trained with online outcome-supervised RL on Numina-3K, a subset of NuminaMath-CoT (Li et al., 2024). For a fair comparison, all trainable baselines are matched to the corresponding model-specific update and prompt-exposure budgets while preserving their method-specific objectives; DEER is evaluated as a training-free inference-time method. Full training budgets and baseline-matching details are provided in Appendix B, with training-cost analysis in Appendix D.

Benchmarks. Our main table reports results on mathematical reasoning and held-out multiple-choice reasoning benchmarks. The mathematical group contains GSM8K (Cobbe et al., 2021), MATH500 (Lightman et al., 2024), AIME2024 (Jia, 2024), and HMMT2025 (MathArena, 2025). Since training uses Numina-3K, we also evaluate out-of-domain generalization on non-math benchmarks: GPQA-Diamond (Rein et al., 2023), ARC-Challenge (Clark et al., 2018), and CommonsenseQA (Talmor et al., 2019).

Decoding and metrics. To isolate method differences without adding sampling variance, all main-table results use greedy single-sample decoding for all methods, with one response per problem and a maximum response length of 16,384 tokens. We report accuracy and average generated response length in tokens. The Avg. column reports example-count-weighted averages across the seven benchmarks. Full decoding and scoring details are provided in Appendix C.

4.2 Main Result

Table 1 shows that RUAR provides a strong accuracy–efficiency trade-off across model scales and benchmark types. Across four backbones and seven benchmarks, RUAR achieves the best or tied-best accuracy in 10 out of 28 settings and the shortest or second-shortest average response length in 24 out of 28 settings. It obtains the highest weighted-average accuracy for DS-7B and Qwen3-8B and the second-highest for DS-1.5B.

On mathematical reasoning benchmarks, RUAR substantially reduces generated tokens while often preserving or improving accuracy. Relative to Base, it improves accuracy on all four mathematical benchmarks for the DS-1.5B model and on three of four for the DS-7B model. For Qwen3-8B, it improves GSM8K and MATH500 accuracy while preserving HMMT2025 accuracy, although the results on Qwen3-1.7B and the remaining hard-math settings are more mixed. Across the four backbones, weighted-average response length is reduced by approximately 28–44% relative to Base.

The efficiency gains also transfer to the held-out non-math benchmarks. RUAR improves all three non-math accuracies over Base for the DS-1.5B and Qwen3-8B models, while preserving or improving them for the DS-7B model. Qwen3-1.7B exhibits small accuracy decreases but still produces substantially shorter responses. Overall, although individual baselines are competitive on particular datasets, RUAR consistently combines competitive accuracy with some of the shortest responses.

4.3 Analysis

We further examine the behavior underlying the gains in Table 1, asking whether RUAR merely shortens reflection or selectively reduces low-utility and post-ready continuation. The utility categories used below are diagnostic labels for analysis, whereas RUAR uses the utility estimate $u_j = p_j^+ - p_j^-$ for advantage rescaling during training.

Low-utility reflection after training. Table 2 applies the answer-forcing utility diagnostic to matched greedy 16K DS-7B MATH500 traces, with each policy's traces re-probed using that same policy. Compared with Base, RUAR produces much shorter responses and substantially fewer reflective reasoning steps, while increasing the helpful-step share from 3.60% to 9.48% (+162.9% relative). Compared with TLMRE, RUAR again produces shorter responses and fewer steps, while increasing the helpful-step share from 7.61% to 9.48% (+24.5% relative). Although the absolute number of helpful steps decreases from 139 to 103 as the total number of steps decreases from 1,826 to 1,087, helpful reflection becomes more concen-

trated. These results suggest that RUAR selectively reduces low-utility continuation rather than uniformly suppressing reflection. Here, Low-util. combines redundant, ineffective, and harmful steps.

Method	Avg. Tok.	#Steps	Helpful	Low-util.
Base	3569	5965	215 (3.60)	5750 (96.40)
PRIME	4352	6313	215 (3.41)	6098 (96.59)
TLMRE	2199	1826	139 (7.61)	1687 (92.39)
RUAR	**2100**	**1087**	103 (**9.48**)	**984 (90.52)**

Table 2: Utility composition on DS-7B MATH500.

Post-ready continuation reduction. Table 3 decomposes the reasoning segment $\boldsymbol{y}_r$ into pre-ready and post-ready regions using the detected answer-ready step. Base and PRIME retain substantial reasoning after becoming answer-ready, while both TLMRE and RUAR greatly reduce this continuation. RUAR produces the fewest pre-ready and post-ready tokens and the lowest post-ready share. Compared with TLMRE, it further reduces post-ready continuation from 204 to 144 tokens while modestly shortening the pre-ready region. This supports the intended behavior of overthinking-mitigating rescaling: RUAR concentrates its reduction in low-utility continuation after answer readiness while also improving overall reasoning efficiency. Appendix L further shows that post-ready reflective steps are predominantly low-utility.

Method	Pre tok.	Post tok.	Post %
Base	2116	1102	34.25
PRIME	2496	1515	37.78
TLMRE	1632	204	11.09
RUAR	**1597**	**144**	**8.27**

Table 3: Pre/post-ready token decomposition on the same traces.

4.4 Ablation Study

Overthinking-mitigating rescaling ablation. Unlike the post-hoc analyses above, Table 4 compares separately trained policies under matched training and evaluation settings. The variants differ only in the ablated answer-ready mechanism and are evaluated on MATH500 using greedy 16K decoding.

The w/o post-ready scaling variant retains answer-ready detection but removes fixed post-ready rescaling by setting $\gamma^{+}_{\text{post}} = \gamma^{-}_{\text{post}} = 1$. Compared with Full RUAR, it produces longer responses and lower accuracy (1,711 versus 1,545 tokens and 78.6% versus 79.4%). The w/o answer-ready step variant instead applies utility-guided rescaling to all selected reflective steps. It achieves slightly higher accuracy than Full RUAR (79.8% versus 79.4%) but requires substantially longer responses (1,966 versus 1,545 tokens).

Full RUAR combines answer-ready detection with fixed post-ready scaling. Its average scaling factors assign less positive credit and preserve more negative-advantage magnitude than either ablation. Overall, Full RUAR provides the strongest accuracy–efficiency balance: it reduces tokens by 21.4% relative to the w/o answer-ready variant with only a 0.4 percentage-point accuracy difference. Additional component and sensitivity results are provided in Appendices I and J.

Variant	Acc.	Avg. Tok.	$\bar{\gamma}^{+}$	$\bar{\gamma}^{-}$
w/o post-ready scaling	78.6	1711	1.024	0.976
w/o answer-ready step	**79.8**	1966	1.020	0.980
Full RUAR	79.4	**1545**	**0.977**	**0.994**

Table 4: Overthinking-mitigating rescaling ablation on MATH500. $\bar{\gamma}^{+}$ and $\bar{\gamma}^{-}$ denote the response-token and training-update averages of the multipliers associated with the $A_{i,t} \geq 0$ and $A_{i,t} < 0$ branches, respectively.

5 Related Work

Efficient reasoning in large reasoning models. Long chain-of-thought reasoning improves test-time problem solving but increases generation cost (Snell et al., 2025; Feng et al., 2025). Existing work therefore reduces the amount, representation, or timing of reasoning. Some methods compress explicit traces through controllable CoT compression, redundant-token pruning, or sketch-style reasoning (Xia et al., 2025; Choi et al., 2025; Aytes et al., 2025). Others restructure the form of reasoning through soft or latent representations (Xu et al., 2025; Zhang et al., 2026; Shi et al., 2026). A third direction adaptively controls reasoning effort by deciding when to think, continue, or exit (Zhang et al., 2025b; Xiang et al., 2026; Zhao et al., 2026; Shen et al., 2025; Lin et al., 2026; Yang et al., 2026). Training-based variants also use length-harmonizing fine-tuning or add length-, difficulty-, reflection-, or budget-aware rewards to favor concise correct answers (Luo et al., 2025; Arora and Zanette, 2026; Hou et al., 2026; Yu et al., 2026b; Chen et al., 2026; Cheng et al., 2026; Liu et al., 2026). These methods mainly control how much

reasoning is generated, represented, or retained. RUAR instead estimates which reflective reasoning steps are useful and uses this utility to rescale advantage signals during training.

Reflective tokens and self-correction. Reflection has been used for iterative feedback, verbal memory, tool-based critique, and deliberative search over reasoning paths (Madaan et al., 2023; Shinn et al., 2023; Gou et al., 2024; Yao et al., 2023). However, self-correction is not uniformly reliable: models can fail to correct errors without external feedback, depend on strong verifiers, misidentify fallacious reasoning, or misjudge their own outputs (Huang et al., 2024; Zhang et al., 2024; Hong et al., 2024; Jiang et al., 2024). Recent work further studies reflection inside long reasoning traces, including verbalized confidence, hidden self-verification, spontaneous correction, post-answer reflection, overused self-verification, reflection-token scheduling, and discourse-marker segmentation (Zeng et al., 2025; Zhang et al., 2025a; Zhao et al., 2025; Kang et al., 2026; Long et al., 2026; Fan et al., 2026; Wang et al., 2025). These findings support our view that reflective tokens such as *Wait* provide useful boundaries for reflective reasoning steps, but are not reliable utility signals.

Process-level signals without process labels. Outcome-based RL trains reasoning models from terminal correctness using policy-gradient objectives, including PPO/REINFORCE-style optimization and recent value-free, group-relative, or sequence-level variants such as RLOO, GRPO, DAPO, and GSPO (Schulman et al., 2017; Ahmadian et al., 2024; Shao et al., 2024; Yu et al., 2026a; Zheng et al., 2025a). Yet terminal rewards assign broad credit to whole rollouts and do not identify which intermediate reasoning steps helped or hurt. Process supervision addresses this with human process labels, automatic step-level labels, process reward models, or process-error benchmarks (Lightman et al., 2024; Wang et al., 2024; Zheng et al., 2025b). Other work reduces explicit process annotation by deriving implicit or learned process rewards from outcome supervision (Yuan et al., 2025; Cui et al., 2026). RUAR shares the goal of obtaining finer-grained process-level signals, but does not train a separate process reward model. Instead, it estimates before/after utility for reflective reasoning steps and converts that utility directly into token-level advantage rescaling.

6 Conclusion

We introduced RUAR, a utility-guided advantage rescaling method for reflective reasoning. RUAR estimates each reflective step's before/after utility with answer-forcing probes and rescales token-level advantages without human process labels or a learned process reward model. It also mitigates overthinking by switching to fixed post-ready rescaling from the earliest answer-ready step onward. Experiments show that RUAR improves the accuracy–efficiency trade-off across mathematical and held-out non-math benchmarks, suggesting that efficient reasoning requires crediting which reflective steps are useful, not only how much the model thinks.

7 Limitations

In our experiments, we instantiate RUAR with compact lexical reflective-token sets centered on *Wait* and *Alternatively*. The selected delimiter configurations reflect backbone-specific reflective-token distributions while keeping step extraction interpretable and computationally efficient. A promising extension is implicit or model-adaptive reflection-step discovery that identifies boundaries from context rather than relying on a fixed lexical inventory.

Answer-forcing adds additional training-time computation during post-training relative to trajectory-level supervision because it estimates reflective-step utility, but introduces no additional inference-time procedure or cost. RUAR obtains this signal without human process labels or a separately learned reward model, and the systems optimizations substantially reduce probing overhead. Further reducing this training-time cost through adaptive probe allocation or lightweight surrogate utility estimation remains an important direction for future work.

8 Acknowledgments

This work was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2020-II201336, Artificial Intelligence Graduate School Support (UNIST)) and by an IITP grant funded by the Korea government (MSIT) (No. RS-2026-25523906, Development of an Industry-Specific Multimodal Hyperscale Foundation Model).

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand. Association for Computational Linguistics.

Daman Arora and Andrea Zanette. 2026. Training language models to reason efficiently. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Simon A. Aytes, Jinheon Baek, and Sung Ju Hwang. 2025. Sketch-of-thought: Efficient LLM reasoning with adaptive cognitive-inspired sketching. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 24296–24320, Suzhou, China. Association for Computational Linguistics.

Weize Chen, Jiarui Yuan, Tailin Jin, Ning Ding, Huimin Chen, Zhiyuan Liu, and Maosong Sun. 2026. The overthinker's DIET: Cutting token calories with DIfficulty-aware training. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Xiaoxue Cheng, Junyi Li, Zhenduo Zhang, Xinyu Tang, Xin Zhao, XinYu Kong, and Zhiqiang Zhang. 2026. Incentivizing dual process thinking for efficient large language model reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Daewon Choi, Jimin Lee, Jihoon Tack, Woomin Song, Saket Dingliwal, Sai Muralidhar Jayanthi, Bhavana Ganesh, Jinwoo Shin, Aram Galstyan, and Sravan Babu Bodapati. 2025. Think clearly: Improving reasoning via redundant token pruning. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 21437–21451, Suzhou, China. Association for Computational Linguistics.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *Preprint*, arXiv:1803.05457.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *Preprint*, arXiv:2110.14168.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, and 6 others. 2026. Process reinforcement through implicit rewards.

Chongyu Fan, Yihua Zhang, Jinghan Jia, Alfred O. Hero, and Sijia Liu. 2026. Cyclicreflex: Improving reasoning models via cyclical reflection token scheduling. In *The Fourteenth International Conference on Learning Representations*.

Sicheng Feng, Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Efficient reasoning models: A survey. *Transactions on Machine Learning Research*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *The Twelfth International Conference on Learning Representations*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025. DeepSeek-R1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.

Ruixin Hong, Hongming Zhang, Xinyu Pang, Dong Yu, and Changshui Zhang. 2024. A closer look at the self-verification abilities of large language models in logical reasoning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 900–925, Mexico City, Mexico. Association for Computational Linguistics.

Bairu Hou, Yang Zhang, Jiabao Ji, Yujian Liu, Kaizhi Qian, Jacob Andreas, and Shiyu Chang. 2026. Thinkprune: Pruning long chain-of-thought of LLMs via reinforcement learning. *Transactions on Machine Learning Research*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*.

Maxwell Jia. 2024. AIME 2024 dataset. `https://huggingface.co/datasets/Maxwell-Jia/AIME_2024`. Dataset.

Dongwei Jiang, Jingyu Zhang, Orion Weller, Nathaniel Weir, Benjamin Van Durme, and Daniel Khashabi. 2024. Self-[in]correct: Llms struggle with discriminating self-generated responses. *Preprint*, arXiv:2404.04298.

Liwei Kang, Yue Deng, Yao Xiao, Zhanfeng Mo, Wee Sun Lee, and Lidong Bing. 2026. First try matters: Revisiting the role of reflection in reasoning models.

Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif

Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. Numinamath. https://huggingface.co/datasets/AI-MO/NuminaMath-CoT.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*.

Weizhe Lin, Xing Li, Zhiyuan Yang, Xiaojin Fu, Hui-Ling Zhen, Yaoyuan Wang, Xianzhi Yu, Wulong Liu, Xiaosong Li, and Mingxuan Yuan. 2026. Trimr: Verifier-based training-free thinking trimming for efficient test-time scaling. In *The Fourteenth International Conference on Learning Representations*.

Wei Liu, Ruochen Zhou, Yiyun Deng, Yuzhen Huang, Junteng Liu, Yuntian Deng, Yizhe Zhang, and Junxian He. 2026. Learn to reason efficiently with adaptive length-based reward shaping. In *The Fourteenth International Conference on Learning Representations*.

Quanyu Long, Kai Jie Jiang, Jianda Chen, Xu Guo, Leilei Gan, and Wenya Wang. 2026. Self-verification dilemma: Experience-driven suppression of overused checking in llm reasoning. *Preprint*, arXiv:2602.03485.

Haotian Luo, Li Shen, Haiying He, Yibo Wang, Shiwei Liu, Wei Li, Naiqiang Tan, Xiaochun Cao, and Dacheng Tao. 2025. O1-Pruner: Length-harmonizing fine-tuning for o1-like reasoning pruning. In *2nd AI for Math Workshop @ ICML 2025*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Thirty-seventh Conference on Neural Information Processing Systems*.

MathArena. 2025. HMMT February 2025. https://huggingface.co/datasets/MathArena/hmmt_feb_2025. Dataset.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A graduate-level google-proof question answering benchmark. *Preprint*, arXiv:2311.12022.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *Preprint*, arXiv:1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *Preprint*, arXiv:2402.03300.

Yi Shen, Jian Zhang, Jieyun Huang, Shuming Shi, Wenjing Zhang, Jiangze Yan, Ning Wang, Kai Wang, Zhaoxiang Liu, and Shiguo Lian. 2025. DAST: Difficulty-adaptive slow-thinking for large reasoning models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 2322–2331, Suzhou (China). Association for Computational Linguistics.

Dachuan Shi, Abedelkadir Asi, Keying Li, Xiangchi Yuan, Leyan Pan, Wenke Lee, and Wen Xiao. 2026. Swireasoning: Switch-thinking in latent and explicit for pareto-superior reasoning LLMs. In *The Fourteenth International Conference on Learning Representations*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2025. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*.

Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota. Association for Computational Linguistics.

Chenlong Wang, Yuanning Feng, Dongping Chen, Zhaoyang Chu, Ranjay Krishna, and Tianyi Zhou. 2025. Wait, we don't need to "wait"! removing thinking tokens improves reasoning efficiency. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 7459–7482, Suzhou, China. Association for Computational Linguistics.

Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. 2024. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, Bangkok, Thailand. Association for Computational Linguistics.

Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. 2025. TokenSkip: Controllable chain-of-thought compression in LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3351–3363, Suzhou, China. Association for Computational Linguistics.

Violet Xiang, Chase Blagden, Rafael Rafailov, Nathan Lile, Sang T. Truong, Chelsea Finn, and Nick Haber. 2026. Just enough thinking: Efficient reasoning with adaptive length penalties reinforcement learning.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. 2025. SoftCoT: Soft chain-of-thought for efficient reasoning with LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 23336–23351, Vienna, Austria. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. 2026. Dynamic early exit in reasoning models. In *The Fourteenth International Conference on Learning Representations*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, YuYue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncai Liu, LingJun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, and 17 others. 2026a. DAPO: An open-source LLM reinforcement learning system at scale. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Zewei Yu, Lirong Gao, Yuke Zhu, Bo Zheng, Junbo Zhao, Sheng Guo, and Haobo Wang. 2026b. Stop unnecessary reflection: Training LRMs for efficient reasoning with adaptive reflection and length coordinated penalty. In *The Fourteenth International Conference on Learning Representations*.

Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. 2025. Free process rewards without process labels. In *Forty-second International Conference on Machine Learning*.

Qingcheng Zeng, Weihao Xuan, Leyang Cui, and Rob Voigt. 2025. Thinking out loud: Do reasoning models know when they're right? In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1394–1407, Suzhou, China. Association for Computational Linguistics.

Anqi Zhang, Yulin Chen, Jane Pan, Chen Zhao, Aurojit Panda, Jinyang Li, and He He. 2025a. Reasoning models know when they're right: Probing hidden states for self-verification. In *Second Conference on Language Modeling*.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025b. AdaptThink: Reasoning models can learn when to think. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3716–3730, Suzhou, China. Association for Computational Linguistics.

Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. 2024. Small language models need strong verifiers to self-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15637–15653, Bangkok, Thailand. Association for Computational Linguistics.

Zhen Zhang, Xuehai He, Weixiang Yan, Ao Shen, Chenyang Zhao, and Xin Eric Wang. 2026. Soft thinking: Unlocking the reasoning potential of LLMs in continuous concept space. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Haoran Zhao, Yuchen Yan, Yongliang Shen, Haolei Xu, Wenqi Zhang, Kaitao Song, Jian Shao, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2026. Let LRMs break free from overthinking via self-braking tuning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Xutong Zhao, Tengyu Xu, Xuewei Wang, Zhengxing Chen, Di Jin, Liang Tan, Yen-Ting Lin, Zishun Yu, Zhuokai Zhao, Yun He, Sinong Wang, Han Fang, Sarath Chandar, and Chen Zhu. 2025. Boosting LLM reasoning via spontaneous self-correction. In *Second Conference on Language Modeling*.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. 2025a. Group sequence policy optimization. *Preprint*, arXiv:2507.18071.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025b. ProcessBench: Identifying process errors in mathematical reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1009–1024, Vienna, Austria. Association for Computational Linguistics.

A RUAR Algorithm

Algorithm 1 summarizes the full RUAR training procedure.

B Training and Baseline Matching Details

Training configuration. All trainable baselines are matched on shared experimental factors, including the backbone initialization, Numina-3K prompt subset and ordering, prompt exposure, and training budget, while preserving each method's defining objective and method-specific components. For online RL methods, we additionally match the rollout count, batch size, optimizer settings, and sequence-length limits. The selected update budgets are 110, 30, 15, and 30 for DS-1.5B, DS-7B, Qwen3-1.7B, and Qwen3-8B, respectively; trainable baselines are matched to these model-specific budgets. Table 5 summarizes the shared online RL configuration. DEER is excluded from this training comparison because it is a training-free inference-time method.

Table 5: Shared online RL hyperparameters.

Item	Value
Batch size	8 prompts
Rollouts per prompt	16
Advantage estimator	RLOO
Learning rate	2×10^{-6}
PPO clip coefficient	0.2
Rollout sampling	Temperature 1.0, top-p 1.0
Maximum response length	16384 tokens
KL penalty	None

RUAR configuration. Table 6 reports the RUAR-specific hyperparameters. The reflective-step extraction rule, utility estimate, answer-ready step definition, and advantage-rescaling equations are defined in Section 3. For the DS backbones, we use $\mathcal{R}_{\text{start}} = \mathcal{R}_{\text{end}} = \{\textit{Wait}\}$. For the Qwen3 backbones, we use $\mathcal{R}_{\text{start}} = \mathcal{R}_{\text{end}} = \{\textit{Wait}, \textit{Alternatively}\}$.

Table 6: RUAR hyperparameters.

Item	Value
Length penalty coefficient λ_{len}	0.2
Answer-forcing samples K	4
Max probe-continuation length A	16 tokens
Answer-ready threshold τ	0.75
Answer-ready streak length c	3
Post-ready positive multiplier γ^{+}_{post}	0.25
Post-ready negative multiplier γ^{-}_{post}	1.25

Training compute. All training runs are conducted using NVIDIA H200 GPUs. DS-1.5B and Qwen3-1.7B use one GPU, while DS-7B and Qwen3-8B use two GPUs.

Baseline implementation. We instantiate the trainable baselines from their publicly released implementations and adapt their model, data, and budget settings to the shared experimental protocol. Each baseline retains its defining objective, optimization procedure, and method-specific constraints; hyperparameters without a direct counterpart in RUAR are kept at the released-code defaults. DEER is applied using its released training-free inference-time procedure.

C Evaluation Details

Evaluation protocol. Table 7 summarizes the decoding hyperparameters used for the main evaluation. All methods use the same single-sample greedy protocol to isolate policy differences without additional sampling variance.

Table 7: Main evaluation decoding hyperparameters.

Item	Value
Number of samples	1
Temperature	0.0
Top-p	1.0
Maximum response length	16384 tokens

D Training-Time Cost Analysis

Answer-forcing introduces additional training-time computation because RUAR estimates reflective-step utility at the boundaries of reflective reasoning steps. This requires short forward continuations and rule-based answer verification, but does not introduce an additional gradient-based backward pass for each reflective reasoning step.

Computational complexity. Let B denote the number of prompts per training update, N the number of rollout trajectories per prompt, and m_i the number of detected reflective reasoning steps in trajectory i, where $i \in \{1, \ldots, BN\}$. Let U_i denote the number of unique before/after boundaries actually probed after within-trajectory deduplication and answer-ready early stopping. Because each reflective reasoning step contributes at most two candidate boundaries, $0 \le U_i \le 2m_i$.

With K answer-forcing samples generated at each probed boundary, the total number of auxiliary

Algorithm 1 RUAR: Utility-Guided Advantage Rescaling

Require: Prompts $\mathcal{X}$, policy π_θ, rule-based final-answer verifier $\mathsf{correct}$; rollout count N, probe count K; answer-forcing prompt $\mathcal{A}$; ready threshold τ, consecutive count c; post-ready scales $\gamma^{+}_{\text{post}}, \gamma^{-}_{\text{post}}$; length coefficient λ_{len}.

for each prompt $x \in \mathcal{X}$ **do**
  Sample N rollouts $\{\boldsymbol{y}^{(i)}\}_{i=1}^{N} \sim \pi_\theta(\cdot \mid x)$
  Score final answers: $z_i \leftarrow \mathsf{correct}(x, \boldsymbol{y}^{(i)})$
  Compute length-regularized rewards $R_i \leftarrow z_i - \lambda_{\text{len}} \rho_i$
  Compute base RLOO advantage signals $A_{i,t}$
  for each rollout $\boldsymbol{y}^{(i)}$ **do**
    Initialize $\gamma_{i,t} \leftarrow 1$ for all response tokens t
    Extract reflective reasoning steps $\mathcal{S}(\boldsymbol{y}^{(i)}) = (\boldsymbol{s}_{i,1}, \ldots, \boldsymbol{s}_{i,m_i})$
    Set $b_i \leftarrow m_i + 1$
    for $j = 1, \ldots, m_i$ **do**
      Set $\boldsymbol{s}^{-}_{i,j} \leftarrow \boldsymbol{y}^{(i)}_{1:a_{i,j}-1}$
      Estimate $p^{-}_{i,j}$ from K samples of $\pi_\theta(\cdot \mid x, \boldsymbol{s}^{-}_{i,j} \oplus \mathcal{A})$ using $\mathsf{correct}$
      if $j \geq c$ and $p^{-}_{i,r} \geq \tau$ for all $r = j - c + 1, \ldots, j$ **then**
        Set $b_i \leftarrow j - c + 1$ and stop probing later steps
        break
      end if
    end for
    for each pre-ready step $\boldsymbol{s}_{i,j}$ with $j < b_i$ **do**
      Set $\boldsymbol{s}^{+}_{i,j} \leftarrow \boldsymbol{y}^{(i)}_{1:e_{i,j}}$
      Estimate $p^{+}_{i,j}$ from K samples of $\pi_\theta(\cdot \mid x, \boldsymbol{s}^{+}_{i,j} \oplus \mathcal{A})$ using $\mathsf{correct}$
      Set utility $u_{i,j} \leftarrow p^{+}_{i,j} - p^{-}_{i,j}$
      for each token $t \in \boldsymbol{s}_{i,j}$ **do**
        $\gamma_{i,t} \leftarrow \max(0, 1 + u_{i,j})$ if $A_{i,t} \geq 0$, else $\max(0, 1 - u_{i,j})$
      end for
    end for
    if $b_i \leq m_i$ **then**
      for each post-ready reasoning token t **do**
        $\gamma_{i,t} \leftarrow \gamma^{+}_{\text{post}}$ if $A_{i,t} \geq 0$, else γ^{-}_{post}
      end for
    end if
    Set $A'_{i,t} \leftarrow \gamma_{i,t} A_{i,t}$ for all response tokens t
  end for
end for
Apply standard advantage normalization to $A'_{i,t}$
Update π_θ using the PPO-style objective with the normalized RUAR-adjusted advantage signal

probe continuations is

$$Q_{\text{probe}} = K \sum_{i=1}^{BN} U_i \le 2K \sum_{i=1}^{BN} m_i.$$

Thus, the number of probe continuations grows linearly with the rollout count, the number of unique probed boundaries, and the number of samples per boundary.

For boundary $u \in \{1, \dots, U_i\}$ in trajectory i, let $l_{i,u}$ denote its prefix length, A the maximum probe-continuation length, and $a_{i,u} \le A$ its actual generated length. Let $P(l)$ denote the ordinary prefix-prefill cost, $\widetilde{P}(l) \le P(l)$ the effective prefill cost after available prefix-cache reuse, and $D(l, a)$ the KV-cached decoding cost of generating a tokens from a prefix of length l. Let $C_{\text{verify}}(q)$ denote the rule-based verification cost for q continuations. Finally, let C_{session} and C_{pool} denote the once-per-update inference-session and verifier-pool setup costs, respectively. The additional probe computation is approximated by

$$\begin{aligned} C_{\text{probe}} \approx\ & C_{\text{session}} + C_{\text{pool}} \\ & + \sum_{i=1}^{BN} \sum_{u=1}^{U_i} \left[\widetilde{P}(l_{i,u}) + KD(l_{i,u}, a_{i,u}) \right] \\ & + C_{\text{verify}} \left(K \sum_{i=1}^{BN} U_i \right). \end{aligned}$$

Let

$$L = \max_{i,u} l_{i,u},$$

$$\bar{U} = \frac{1}{BN} \sum_{i=1}^{BN} U_i,$$

$$\bar{m} = \frac{1}{BN} \sum_{i=1}^{BN} m_i.$$

Because $U_i \le 2m_i$, we have $\bar{U} \le 2\bar{m}$. Excluding the once-per-update setup terms and rule-based verification, the probe cost can be summarized as

$$C_{\text{probe}} = O\big(BN\bar{U}\,[P(L) + KD(L, A)]\big).$$

Longer reasoning chains increase computation through both longer prefixes and potentially more reflective reasoning steps. Under standard dense attention, $P(L) = O(L^2)$ and $D(L, A) = O(AL + A^2)$, giving the worst-case attention cost

$$O\big(BN\bar{U}\,\big[L^2 + K(AL + A^2)\big]\big),$$

with model-dimension constants omitted.

In our main configuration, $B = 8$, $N = 16$, $K = 4$, and $A = 16$. Therefore, each update generates

$$Q_{\text{probe}} = KBN\bar{U} = 512\bar{U}$$

auxiliary continuations, corresponding to at most

$$AQ_{\text{probe}} = 8{,}192\bar{U}$$

newly generated probe tokens.

Systems efficiency and empirical cost. To control this cost, the implementation deduplicates coincident boundaries, batches continuations across active trajectories, and stops further probing after detecting a stable answer-ready step. It also retains the inference session and prefix cache across ordered probing rounds and reuses a persistent verifier pool, amortizing weight synchronization, engine initialization, and verifier setup across the complete probe phase.

Method	Process-level signal	Separate RM	GPU-s/ update
TLMRE	No	No	654.6
RUAR	Yes	No	960.2
PRIME	Yes	Yes	1438.4

Table 8: Allocated training compute per update on Qwen3-8B, averaged over updates 1–30 on two H200 GPUs.

RUAR's additional cost over TLMRE reflects its estimation of reflective-step utility. It nevertheless requires less allocated GPU time than PRIME while avoiding a separately learned reward model. Answer-forcing is used only during post-training and introduces no additional inference-time procedure or cost.

E Length-Regularized Terminal Reward

For a fixed prompt x, let $L_i = |\boldsymbol{y}^{(i)}|$ be the generated response length and let $z_i = z(x, \boldsymbol{y}^{(i)})$ denote final-answer correctness. Following Arora and Zanette (2026), we use the terminal reward

$$R_i = z_i - \lambda_{\text{len}} \rho_i,$$

where ρ_i is a response-level length penalty and λ_{len} controls its strength.

The penalty is applied only to correct rollouts. This encourages efficiency among successful solutions without rewarding short incorrect attempts.

Among correct rollouts for the same prompt, let μ_L and σ_L denote the mean and standard deviation of their lengths. We compute

$$\rho_i = z_i \cdot \mathrm{sigmoid}\left(\frac{L_i - \mu_L}{\max(\sigma_L, \epsilon)}\right),$$

where ϵ is a small constant used only for numerical stability. If a prompt has no correct rollout, the length penalty is set to zero for all rollouts of that prompt.

Since ρ_i is multiplied by z_i, incorrect rollouts receive $\rho_i = 0$ and therefore $R_i = 0$. For correct rollouts, the reward remains near one but is reduced more for responses that are longer than other correct rollouts for the same prompt.

F Answer-Forcing Prompt and Verification Details

This appendix describes how answer-forcing probes are generated and verified. For a probe point corresponding to a before-step or after-step partial trace $\boldsymbol{y}_{1:q}$, we append the answer-forcing prompt

$$\mathcal{A} = \texttt{\textbackslash n**Final Answer**\textbackslash n\textbackslash boxed}.$$

The probe input to the policy is the original prompt followed by the partial trace and $\mathcal{A}$. In our experiments, we use $K = 4$ and sample short final-answer continuations from the current policy, with each continuation capped at 16 new tokens.

For verification, each sampled continuation is concatenated back to the probed partial trace, yielding a completion of the form

$$\boldsymbol{y}_{1:q} \oplus \mathcal{A} \oplus o_k.$$

This completion is passed to the same rule-based final-answer scorer used for terminal outcome scoring, together with the gold answer and data-source metadata. The scorer extracts a candidate final answer from the completion and compares it against the gold answer. In our math setting, the extractor prioritizes the final boxed answer when present and otherwise falls back to standard answer markers; the extracted candidate is then normalized and checked for mathematical equivalence with the gold answer. The verifier returns 1 if the extracted answer is judged equivalent to the gold answer and 0 otherwise.

For each probe point, the correct-answer probability is estimated by averaging the K binary verification results. Thus, p_j^- and p_j^+ are Monte Carlo estimates of whether the before-step and after-step partial traces can already lead to the correct final answer under the answer-forcing prompt. These probe continuations are used only for estimating the step utility $u_j = p_j^+ - p_j^-$; they are not treated as additional RL rollouts and do not provide process labels or learned process rewards.

G Reflective-Token Inventory and Delimiter Choice

RUAR requires candidate reflective-step boundaries at which before/after answer-forcing probes can be applied, but it is not intrinsically tied to specific reflective tokens such as *Wait* or *Alternatively*. Importantly, these tokens are not themselves treated as utility signals; they are used only to identify candidate reflective reasoning steps, while the utility of each step is determined through before/after answer-forcing probes. In the evaluated reasoning models, explicit reflective tokens provide a practical operationalization of such step boundaries. They occur frequently in long reasoning traces and typically signal reconsideration, verification, or a shift in strategy. They therefore provide semantically interpretable checkpoints for measuring changes in answer readiness, without probing every token or introducing arbitrary fixed-interval boundaries.

To characterize the availability and backbone-specific distribution of explicit reflective-boundary cues, we analyze the complete thinking regions of MATH500 Base traces generated under greedy decoding with a 32,768-token response limit for all four evaluated backbones. The diagnostic follows the positional scope of the RUAR extractor without requiring paragraph- or sentence-initial occurrence. We compare exact, case-sensitive whole-word occurrences of the delimiter tokens *Wait* and *Alternatively* with a broader inventory of reflective expressions. For a casing-matched comparison, all remaining expressions are also required to begin with their canonical uppercase form.

Table 9 presents the expanded lexical inventory used in this diagnostic. It includes representative expressions associated with reflection, strategy shifts, and verification beyond the two delimiter tokens used by RUAR. Overlapping lexical matches are counted once.

Table 10 summarizes the resulting distribution. Within the expanded inventory, exact *Wait* and *Alternatively* together account for 69.5–73.2% of lexi-

Role	Expression family	Canonical expressions
Reflection	*Wait*	*Wait*
	Hesitation	*Hmm*; *Hold on*; *Not sure*
	Error detection	*Maybe/Perhaps I made a mistake/error*
	Recheck	*Just to make sure*; *Just to be thorough*
	Check/verify	*Let me/Let's check, verify, recheck, reconsider, rethink, confirm, or test*
	Conflict detection	*Which one is correct?*; *Which is the correct answer?*
Strategy	*Alternatively*	*Alternatively*
	Correction	*Actually*; *But actually*
	Alternative approach	*Another/A different way or approach*
Verification	Checks out	*That/This/It checks out*
	No mistakes found	*I do not see/find any mistakes/errors*
	Confirmation	*Yes/So, that's correct*

Table 9: Expanded casing-matched lexical inventory used for the reflective-token distribution diagnostic. Common discourse prefixes such as *But*, *So*, and *Now* are included where applicable.

cal matches across all four backbones. The relative composition is backbone-dependent. The DS models are more strongly *Wait*-dominant, with *Wait* accounting for 57.8–60.3% and *Alternatively* for 12.9–13.6%. In contrast, the Qwen3 models assign less mass to *Wait* (45.7–47.9%) and substantially more to *Alternatively* (22.4–23.8%).

Backbone	Total	*Wait*	*Alt.*	Other	*Wait+Alt.*
DS-1.5B	10,197	5,895 (57.8)	1,384 (13.6)	2,918 (28.6)	**7,279 (71.4)**
DS-7B	11,159	6,727 (60.3)	1,440 (12.9)	2,992 (26.8)	**8,167 (73.2)**
Qwen3-1.7B	18,139	8,688 (47.9)	4,057 (22.4)	5,394 (29.7)	**12,745 (70.3)**
Qwen3-8B	17,893	8,174 (45.7)	4,253 (23.8)	5,466 (30.5)	**12,427 (69.5)**

Table 10: Reflective-token distribution in MATH500 Base traces. Parentheses report percentages within the expanded inventory in Table 9. Here, *Alt.* stands for *Alternatively*.

These results establish that the selected reflective tokens provide frequent candidate boundaries and reveal clear backbone-dependent differences in their usage, but frequency alone does not determine the start/end configuration. We therefore combine this diagnostic with the delimiter sensitivity analysis in Appendix J, which evaluates the corresponding delimiter choices. Accordingly, we use $\mathcal{R}_{\text{start}} = \mathcal{R}_{\text{end}} = \{\textit{Wait}\}$ for the DS backbones and $\mathcal{R}_{\text{start}} = \mathcal{R}_{\text{end}} = \{\textit{Wait}, \textit{Alternatively}\}$ for the Qwen3 backbones.

We do not aim to maximize lexical coverage by indiscriminately including every expression in Table 9. Broader reflective expressions may provide less specific step boundaries, while each additional start delimiter can introduce new reflective reasoning steps and corresponding answer-forcing probes. Delimiter selection therefore reflects a practical trade-off among lexical coverage, boundary specificity, and the computational cost of answer-forcing. The reported percentages are conditional shares within the expanded inventory rather than estimates of recall over all possible reflection events.

The choice of delimiter tokens is an operational component rather than a fixed requirement of RUAR. Models that express reflection through different surface cues could use delimiter sets aligned with their own reasoning traces. For models without stable lexical reflection cues, the same core utility-estimation and advantage-rescaling framework could in principle be paired with a contextual or model-adaptive reflective-step boundary detector. The utility definition itself depends on the before/after effect of an extracted reflective reasoning step, rather than on the literal surface form of its delimiter.

H Answer-Ready Step Analysis

Answer-ready step detection during training. RUAR uses the answer-ready step to identify when a partial reasoning trace is already likely to produce the correct final answer under answer-forcing. Table 11 summarizes its detection frequency over the 150-update trajectory from which the s110 checkpoint is selected. Each update contains 8 prompts and 16 rollouts per prompt, yielding 128 rollout responses.

Updates	Detect.	Count128	Avg. Tok.
All	7.3	9.3	3908
1–25	23.1	29.6	6108
26–50	8.9	11.4	5177
51–75	5.4	6.9	4330
76–100	3.0	3.8	3105
101–125	2.2	2.8	2465
126–150	1.2	1.6	2265

Table 11: Answer-ready step detection during DS-1.5B RUAR training. Detect. is the percentage of rollout responses with a detected answer-ready step, and Count128 is the corresponding average count per update.

The answer-ready step is detected selectively but consistently. Across the complete run, it appears in 7.3% of rollout responses, corresponding to 9.3

responses per update. The detection rate decreases steadily from 23.1% during updates 1–25 to 1.2% during updates 126–150, alongside a reduction in average response length from 6,108 to 2,265 tokens. This behavior is consistent with the intended role of the mechanism: once an answer-ready step is detected, subsequent reflective reasoning steps are treated as post-ready and receive fixed rescaling. As the policy becomes more concise, fewer responses require this intervention.

I Component Ablation

Table 12 ablates RUAR's training signals on MATH500. Plain RLOO performs below the Base model, indicating that online RL alone does not explain the gains. The length-only variant corresponds to TLMRE, using the length penalty without reflective-step utility rescaling. It substantially improves accuracy while shortening responses. Reflective-step utility rescaling alone also improves accuracy over Base, showing that the reflective-step utility signal is useful without length regularization. Full RUAR achieves the highest accuracy and the shortest responses among the trained variants, indicating that reflective-step utility rescaling complements the length objective.

Variant	RLOO	Len.	Refl. util.	Acc.	Avg. Tok.
Base	–	–	–	67.6	3519
RLOO only	✓	–	–	62.8	3538
Length only	✓	✓	–	79.2	1876
Reflection only	✓	–	✓	78.6	3533
Full RUAR	✓	✓	✓	**80.0**	**1773**

Table 12: Component ablation on DeepSeek-R1-Distill-Qwen-1.5B MATH500. All trained variants use the same 110-update budget and greedy 32K evaluation protocol.

J Additional Sensitivity Analyses

Unless otherwise stated, all sensitivity analyses use the DS-1.5B backbone, 110-update checkpoints, greedy single-sample decoding on MATH500, and a 32768-token response limit.

J.1 Length-Penalty Coefficient Sensitivity

We evaluate the sensitivity of RUAR to the length-penalty coefficient λ_{len}. The $\lambda_{\text{len}} = 0.0$ condition disables only the response-level length penalty while retaining reflective-step utility rescaling.

The default coefficient $\lambda_{\text{len}} = 0.2$ achieves the highest accuracy while substantially shortening responses. Increasing it to 0.3 further reduces response length but causes a large accuracy drop, indicating over-compression.

λ_{len}	Acc.	Correct	Avg. Tok.
0.0	78.6	393/500	3533
0.1	78.6	393/500	1972
0.2	**80.0**	**400/500**	1773
0.3	71.0	355/500	**797**

Table 13: Length-penalty coefficient sensitivity on MATH500.

J.2 Reflective-Token Sensitivity

We examine whether RUAR depends strongly on the reflective tokens used as step-start and step-end delimiters. We compare three configurations under matched fixed-update training: *Wait*-to-*Wait*, *Wait*-to-*Wait/Alternatively*, and *Wait/Alternatively*-to-*Wait/Alternatively*. DS-1.5B uses 110 updates, DS-7B and Qwen3-8B use 30 updates, and Qwen3-1.7B uses 15 updates.

	Wait→*Wait*		*Wait*→*Wait/Alt.*		*Wait/Alt.*→*Wait/Alt.*	
Model	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.
DS-1.5B	**79.4**	**1545**	78.6	1639	77.2	1765
DS-7B	**90.0**	2100	89.4	2465	89.8	**2023**
Qwen3-1.7B	87.2	**3398**	86.0	3502	**87.6**	3450
Qwen3-8B	**94.0**	3079	93.6	2991	93.6	**2920**

Table 14: Reflective-token sensitivity on MATH500 under matched fixed-update training and greedy 16K evaluation.

The delimiter preference differs across backbone families. For both DS models, using *Wait* as both the step-start and step-end delimiter achieves the highest accuracy. For Qwen3-1.7B, using both *Wait* and *Alternatively* as starts and endpoints achieves the highest accuracy. On Qwen3-8B, the same configuration produces the shortest responses, with only a 0.4 percentage-point accuracy difference from the highest-accuracy configuration. These results support the backbone-specific delimiter configurations used in the main experiments: *Wait*-to-*Wait* for the DS models and *Wait/Alternatively*-to-*Wait/Alternatively* for the Qwen3 models. Overall, however, the accuracy differences across delimiter configurations remain modest (at most 2.2 percentage points), suggesting that RUAR is reasonably robust to delimiter choice while benefiting slightly from backbone-aligned configurations.

K Utility-Composition Diagnostic Details

For the utility-composition analysis in Table 2, we apply the same answer-forcing probe procedure as in Section 3.2. For each extracted target reflective reasoning step s_j, we estimate p_j^- and p_j^+ from the before-step and after-step partial traces, respectively. Each policy's traces are re-probed using that same policy. During training, RUAR uses the utility estimate $u_j = p_j^+ - p_j^-$ directly for advantage rescaling. In this diagnostic analysis, we additionally assign threshold-based labels to summarize how each step changes the answer-readiness status of the partial trace.

We classify each step by thresholding both estimates at the answer-ready threshold $\tau = 0.75$:

	$p_j^+ < \tau$	$p_j^+ \geq \tau$
$p_j^- < \tau$	*Ineffective*	*Helpful*
$p_j^- \geq \tau$	*Harmful*	*Redundant*

Thus, helpful steps move a partial trace from not answer-ready to answer-ready, redundant steps remain answer-ready before and after the step, ineffective steps remain not answer-ready, and harmful steps move a partial trace from answer-ready to not answer-ready.

All extracted target reflective reasoning steps are probed in this diagnostic; no answer-ready early stopping is applied during probing. Step endpoints follow the DS-7B extraction rule used in the corresponding main experiment: each target step starts at a *Wait* token and ends immediately before the next *Wait*, or at the end of the reasoning segment if no later *Wait* appears.

Table 15 reports response lengths, exact case-sensitive occurrences of *Wait* and *Alternatively* within the reasoning segment, and extracted step counts. Because *Wait* defines both the start and endpoint delimiters in this analysis, each detected *Wait* initiates one extracted reflective reasoning step; therefore, the *Wait* and step counts coincide. *Alternatively* is reported descriptively and is not used as a delimiter in this analysis.

Table 16 reports the corresponding full utility composition. Compared with Base, RUAR increases the helpful-step share from 3.60% to 9.48% while reducing the low-utility share from 96.40% to 90.52%. Compared with TLMRE, it increases the helpful-step share from 7.61% to 9.48%. The absolute number of harmful steps decreases from 62 for Base and 31 for TLMRE to 18 for RUAR, corresponding to reductions of 71.0% and 41.9%, respectively. Harmful steps remain rare across all methods, so their percentages should be interpreted together with the total number of extracted steps.

Method	Avg. Tok.	#*Wait*	#*Alt.*	#Steps
Base	3569	5965	1538	5965
PRIME	4352	6313	2437	6313
TLMRE	2199	1826	707	1826
RUAR	**2100**	**1087**	933	**1087**

Table 15: Response length, reflective-token occurrences, and extracted reflective reasoning step counts on DS-7B MATH500 under greedy 16K evaluation.

Method	Helpful	Redundant	Ineffective	Harmful
Base	215 (3.60)	2795 (46.86)	2893 (48.50)	62 (1.04)
PRIME	215 (3.41)	3077 (48.74)	2954 (46.79)	67 (1.06)
TLMRE	139 (7.61)	613 (33.57)	1043 (57.12)	31 (1.70)
RUAR	103 (**9.48**)	**348 (32.01)**	**618 (56.85)**	**18 (1.66)**

Table 16: Full utility composition of extracted reflective reasoning steps on DS-7B MATH500 under greedy 16K evaluation. Parentheses report percentages.

L Pre/Post-Ready Utility Breakdown

To complement the post-ready continuation analysis in Table 3, we further decompose extracted target reflective reasoning steps by the diagnostic utility category introduced in Appendix K, before and after the detected answer-ready step. For a response with answer-ready step index b, steps with $j < b$ are counted as pre-ready, while steps with $j \geq b$ are counted as post-ready. If no answer-ready step is detected, all extracted target reflective reasoning steps are counted as pre-ready and there is no post-ready continuation.

We use the same answer-forcing diagnostic as in Appendix K. These utility categories are used only as diagnostic labels for analysis; during training, RUAR uses the utility estimate $u_j = p_j^+ - p_j^-$ directly for advantage rescaling. Percentages in Table 17 are computed within each method–region pair. The post-ready rows are dominated by low-utility steps across methods, supporting the interpretation that post-ready continuation is mostly confirmatory or ineffective rather than corrective. RUAR reduces the number of post-ready target reflective reasoning steps from 2,693 for Base and 402 for TLMRE to 210, while most remaining post-ready steps are still low-utility.

Method	Region	#Steps	Helpful	Low-util.
Base	Pre-ready	3272	190 (5.81)	3082 (94.19)
Base	Post-ready	2693	25 (0.93)	2668 (99.07)
PRIME	Pre-ready	3156	178 (5.64)	2978 (94.36)
PRIME	Post-ready	3157	37 (1.17)	3120 (98.83)
TLMRE	Pre-ready	1424	129 (9.06)	1295 (90.94)
TLMRE	Post-ready	402	10 (2.49)	392 (97.51)
RUAR	Pre-ready	**877**	99 (**11.29**)	**778 (88.71)**
RUAR	Post-ready	**210**	4 (1.90)	**206 (98.10)**

Table 17: Pre/post-ready utility composition of extracted reflective reasoning steps on DS-7B MATH500 under greedy 16K evaluation. Percentages are computed within each method–region pair; Low-util. combines redundant, ineffective, and harmful steps.

M Case Study: Reflection Step Utility and Answer-Ready Step

We provide a method-level case study of RUAR on a real MATH500 rollout from the Qwen3-8B step-probe diagnostic, using MATH500 sample index 220. The completed rollout is correct but long: it contains 14612 response tokens and 41 *Wait* reflective tokens. For each extracted target reflective reasoning step, we probe its before-step and after-step partial traces. Concretely, for a partial trace h, we append the fixed answer-forcing prompt `\n**Final Answer**\n\boxed` and sample $K = 4$ short final-answer continuations from the current policy, with a maximum of 16 new tokens. This prompt does not reveal the gold answer; it only forces the model to stop reasoning and complete a boxed final answer. Each completed probe is scored by the rule-based final-answer verifier against the gold answer, and the correct-answer probability estimate is the fraction of the four probes that are correct.

Since this example illustrates a correct high-reward trajectory, the table describes the scaling applied to positive-advantage tokens; the symmetric negative-advantage case is defined in Section 3.3. After three consecutive near answer-ready steps with $p^- \geq \tau = 0.75$, RUAR identifies the first step in that consecutive run as the answer-ready step and applies fixed post-ready rescaling from that step onward within the reasoning segment $\boldsymbol{y}_r$. Table 18 summarizes how RUAR assigns pre-ready step utilities, identifies the answer-ready step, and switches to fixed post-ready rescaling in this rollout.

N Case Study: Reducing Redundant Reflective Continuation

We provide a case study on GSM8K example `gsm8k-test-97` using the DS-1.5B policies, where both Base and RUAR reach the correct answer but differ substantially in how much redundant reflective continuation they generate. The gold answer is $\boxed{12}$. Base uses 2462 response tokens with 11 extracted target reflective reasoning steps, while RUAR uses 329 response tokens with 1 extracted target reflective reasoning step. The traces initially follow the same solution path, but Base repeatedly reopens the solved reasoning state after *Wait* and temporarily considers a misleading 6-tomato shortcut. In Table 19, the green-highlighted step marks the Base reflective reasoning step that increases correct-answer probability ($p^- = 0.00 \rightarrow p^+ = 1.00$). The final row summarizes the answer-forcing results for all extracted target reflective reasoning steps in this example.

Stage	Rollout excerpt	p^-	p^+	RUAR action
Problem	"Two numbers, x and y, are selected at random from the interval $(0, 3)$. What is the probability that a triangle with sides of length 1, x, and y exists?" Gold answer: $\boxed{\frac{1}{2}}$.			
Initial partial trace	"… The triangle inequalities are $1 + x > y$, $1 + y > x$, and $x + y > 1$ … visualize this as a region in the coordinate plane where $x, y \in (0, 3)$ … the total area is $3 \times 3 = 9$ …"	–	–	–
Step 1	"*Wait*, no. Let me check … $y = 1 + x$ and $x = 1 + y$, rewritten as $y = x - 1$ … so these two lines are parallel?"	0.00	0.00	Neutral step: $u = 0.00$. Positive credit is unchanged ($\gamma = 1 + u = 1.00$).
Step 2	"*Wait*, both have slope 1 … the region between them is bounded by $y = x + 1$ and $y = x - 1$ inside the square …"	0.00	0.50	$u = +0.50$. This step increases the correct-answer probability, so positive credit in the pre-ready step is amplified by $\gamma = 1 + u = 1.50$.
Step 3	"*Wait*, but if we have both $y < 1 + x$ and $x < 1 + y$, then combining these gives $\lvert x - y \rvert < 1$ … area between these two lines within the square …"	0.50	0.00	$u = -0.50$. Although the rollout later recovers, this step decreases the correct-answer probability at this partial trace, so positive credit in the step is reduced to $\gamma = 1 + u = 0.50$.
Step 4	"*Wait*, actually, the area of this triangle is (base $\times$ height)$/2$ … maybe it is easier to think of it as a right triangle."	0.00	0.00	Neutral step: $u = 0.00$. The step does not change the local correct-answer probability estimate, so positive credit is left unchanged.
Step 5	"*Wait*, when $y \geq x + 1$ … the area above this line is a triangle … area 2 … similarly, the area where $y \leq x - 1$ is also 2 … so $\lvert x - y \rvert < 1$ has area $9 - 4 = 5$ … subtract $x + y \leq 1$, area 0.5 … probability $4.5/9 = \frac{1}{2}$."	0.00	1.00	$u = +1.00$. This step makes the partial trace likely to produce the correct answer, so positive credit is strongly amplified ($\gamma = 2.00$).
Ready probe 1	"*Wait*, but let me verify this again … total area where $\lvert x - y \rvert < 1$ is 5 … area where $\lvert x - y \rvert < 1$ and $x + y \leq 1$ is 0.5 … probability is $4.5/9 = \frac{1}{2}$ …"	1.00	–	First near answer-ready step. Since $p^- \geq \tau = 0.75$, RUAR starts a consecutive run and records this step as the candidate answer-ready step. If the run reaches length $c = 3$, this candidate is confirmed as the answer-ready step, and this step itself is included in the post-ready region.
Ready probe 2	"*Wait*, maybe I can calculate the area directly … the conditions are $\lvert x - y \rvert < 1$ and $x + y > 1$ …"	≥ 0.75	–	Second consecutive near answer-ready step. The consecutive run continues; once the run is confirmed, this step is treated as post-ready continuation rather than a pre-ready utility-guided step.
Ready probe 3	"*Wait*, perhaps it is a polygon with vertices $(0, 1)$, $(1, 0)$, $(3, 2)$, and $(2, 3)$ …"	≥ 0.75	–	Third consecutive near answer-ready step. The candidate from Ready probe 1 is confirmed as the answer-ready step, so Ready probe 1 and all later reasoning-segment tokens receive fixed post-ready scaling, while the answer segment $\boldsymbol{y}_a$ is excluded.
Post-ready continuation	"… Alternatively, maybe I can calculate the area directly … *Wait*, perhaps it is a polygon … *Wait*, no … let me go back to the subtraction method … $9 - 4 = 5$ … subtract 0.5 … probability $\frac{1}{2}$ …" *Answer segment* $\boldsymbol{y}_a$ "… Final answer: $\boxed{\frac{1}{2}}$."	–	–	All reasoning-segment tokens from the answer-ready step onward are treated as post-ready continuation. For this correct trajectory, positive credit over the gray region is reduced ($\gamma^+_{\text{post}} = 0.25$), while the answer segment $\boldsymbol{y}_a$ is excluded from fixed post-ready rescaling.

Table 18: Method-level case study from Qwen3-8B on MATH500 sample 220. The same rollout contains helpful, harmful, and answer-ready reflective reasoning steps. Gray text denotes post-ready continuation; the answer segment $\boldsymbol{y}_a$ is excluded from fixed post-ready rescaling.

Stage	Base (2462 tok., 11 target steps)	RUAR (329 tok., 1 target step)
Problem	"Freda is cooking canned tomatoes into sauce. She loses half the volume when cooking. Each 16-ounce can contains three tomatoes. Her last batch made 32 ounces of sauce. How many tomatoes did she use?" Gold answer: $\boxed{12}$.	
Problem setup (238 vs. 54 tok.)	"Okay, so Freda is cooking canned tomatoes into sauce, and she loses half the volume when she does that. Each can she uses has three tomatoes, and her last batch made 32 ounces of sauce. I need to figure out how many tomatoes she used. Hmm, let me break this down step by step. First, let's understand the process. She starts with some number of cans of tomatoes. Each can has three tomatoes. When she cooks the tomatoes into sauce, she loses half the volume . . . "	"Okay, let me try to figure out how many tomatoes Freda used. Hmm, so she cooks canned tomatoes into sauce and loses half their volume. Each can has three tomatoes and is 16 ounces. Her last batch made 32 ounces of sauce."
Correct path reached (green step: Base p^-=0.00 → p^+=1.00)	"*Wait*, so if she uses c cans, the total volume of tomatoes is $16c$ ounces. Then, when she cooks them, she loses half the volume, so the sauce made is $(16c)/2 = 8c$ ounces. But she made 32 ounces of sauce. So, $8c = 32$. Therefore, $c = 32/8 = 4$. So, she used 4 cans of tomatoes. Since each can has 3 tomatoes, the total number of tomatoes is $4 \times 3 = 12$."	"First, I need to find out how much tomato volume she started with before losing half. Since she ended up with 32 ounces after losing half, that means the original volume was double that. So, $32 \times 2 = 64$ ounces. Now, each can is 16 ounces . . . $64/16 = 4$ cans. Each can has three tomatoes, so $4 \times 3 = 12$ tomatoes."
After deriving the answer (1765 vs. 76 tok.)	"*Wait*, that seems straightforward, but let me double-check . . . . . . She made 32 ounces of sauce. Since the sauce is half the volume of the tomatoes, the total volume of tomatoes she used was $32 \times 2 = 64$ ounces . . . $64/16 = 4$ cans . . . $4 \times 3 = 12$ tomatoes. Alternatively, maybe thinking about the number of tomatoes per ounce? Each can has 3 tomatoes and 16 ounces, so $3/16$ tomatoes per ounce. Then, 32 ounces would be $32 \times (3/16) = 6$ tomatoes. *Wait*, that's different. Hmm, that can't be right because 6 is not 12. *Wait*, let me recast that . . . But that contradicts the previous answer. So, which one is correct? *Wait*, maybe I messed up the ratio . . . 32 ounces of sauce is 64 ounces of tomatoes . . . 4 cans . . . 12 tomatoes. . . . *Wait*, but let me think about the initial problem again . . . *Wait*, but let me think about the ratio again . . . So, the answer is 12 tomatoes."	"*Wait*, let me check that again. Each can is 16 ounces, and she used 4 cans, so $4 \times 16 = 64$ ounces. She started with 64 ounces, lost half, so 32 ounces made. Each can has 3 tomatoes, so $4 \times 3 = 12$ tomatoes. Yeah, that seems right."
Conclusion (277 vs. 80 tok.)	"Freda used $\boxed{12}$ tomatoes. . . . Let c be the number of cans used. The total volume of tomatoes is $16c$ ounces. When cooking, the volume of sauce is half of this, so $8c = 32$, hence $c = 4$. Each can contains 3 tomatoes, so $4 \times 3 = 12$. Thus, Freda used $\boxed{12}$ tomatoes."	"Freda used $\boxed{12}$ tomatoes. Freda started with 64 ounces of tomato volume, which came from 4 cans. Each can contains 3 tomatoes, so she used $4 \times 3 = 12$ tomatoes. $\boxed{12}$"
Diagnostic summary (all target steps)	**11 target steps:** 1 helpful, 10 redundant, 0 ineffective, 0 harmful. The green-highlighted reflective reasoning step changes correct-answer probability from $p^- = 0.00$ to $p^+ = 1.00$. The remaining 10 target steps are redundant ($p^- = p^+ = 1.00$), showing that Base repeatedly continues after the partial trace is already answer-ready.	**1 target step:** 0 helpful, 1 redundant, 0 ineffective, 0 harmful. The single target step is a short redundant verification ($p^- = p^+ = 1.00$). Thus, RUAR reduces repeated redundant post-ready continuation from 10 steps to 1 step.

Table 19: Qualitative comparison on GSM8K example `gsm8k-test-97`. Both methods answer correctly, but Base repeatedly reopens the solved reasoning state after deriving the answer and temporarily considers a misleading 6-tomato shortcut. The green-highlighted step marks the Base reflective reasoning step that increases correct-answer probability ($p^- = 0.00 \rightarrow p^+ = 1.00$); the final row shows that the remaining 10 Base target steps are redundant, while RUAR reduces this repeated redundant continuation to one short verification step.